

Veri Dönüşümü

Görsel Programlama - Hafta 6



HAFTA 6: Veri Dönüşümü ve Hata Yönetimi

Bu sunum, Bilişim Güvenliği Teknolojisi kapsamındaki Görsel Programlama dersi için hazırlanmıştır.

Ana tema: Veri Dönüşümü ve Hata Yönetimi: Try-Catch.



Dersin Amacı ve Haftanın Konusu

Bu hafta, öğrencilerin programlamanın temel yapı taşları olan verileri doğru işlemesi üzerine odaklanılacaktır.
Konular, veri tipleri arasındaki dönüşüm mekanizmalarını ve programın güvenilirliğini artırmayı içermektedir.

Öğrenciler, farklı veri tipleri arasında dönüşüm yapmanın neden gerekli olduğunu kavrayacaklardır.
Özellikle metin (string) tiplerinden sayısal tiplere geçişin kritikliği analiz edilecektir.

Pedagojik Hedefler (2/3)

Öğrenciler, veri dönüşümlerini gerçekleştirmek için C# programlama dilinde Convert sınıfını ve Parse metotlarını etkin bir şekilde kullanabilme yetkinliği kazanacaklardır.

Kullanıcı hatalarından kaynaklanan program çökmelerini (exception) engellemek hedeflenmektedir.
Temel try-catch yapısının anlaşılması ve pratik uygulamalarda kullanılması sağlanacaktır.



Veri Tipi Dönüşümünün Gerekliliği

Veri Tipi Dönüşümü (Type Casting veya Conversion), bir tipteki verinin başka bir tipe çevrilmesi işlemidir.
Bu, özellikle kullanıcı arayüzlerinden alınan verilerin işlenmesi sırasında elzemdir.



TextBox Verisinin Doğası

Önceki haftanın hesap makinesi projesi bağlamında, TextBox kontrolünün 'Text' özelliğinin daima 'string' (metin) tipinde veri döndürdüğü hatırlanmalıdır. Bu, kullanıcının sayı girse bile sistemin onu metin olarak algıladığı anlamına gelir.

Matematiksel İşlem Kısıtlaması

Bir programın matematiksel işlemler yapabilmesi için, 'string' tipindeki verinin mutlaka sayısal bir tipe dönüştürülmesi gerekmektedir. Örneğin, '25' stringi, matematiksel toplama yapılabilmesi için 25 tamsayısına (int) veya ondalıklı sayıya (double) çevrilmelidir.



Tercüman Analojisi

Bu dönüşüm süreci, farklı dilleri konuşan iki insanın anlaşması için bir tercümana ihtiyaç duymasına benzetilebilir.
Dönüştürücü metotlar, programlama ortamında bu 'tercüman' rolünü üstlenir.



Dönüşümde Kullanılan Temel Sınıf

C# dilinde veri tipi dönüşümleri için kullanılan en yaygın ve genel yöntemlerden biri 'Convert' sınıfıdır.
Bu sınıf, geniş bir tip yelpazesi için dönüşüm fonksiyonları sunar.



Convert Metotları: Tamsayı Dönüşümü

`Convert.ToInt32(deger)` metodu, kendisine verilen veriyi 32-bit bir tam sayıya (`int`) çevirir.

32-bit standardı, tamsayı depolama kapasitesini belirtir ve genellikle yeterli aralık sağlar.

Convert Metotları: Ondalık Dönüşüm

`Convert.ToDouble(deger)` metodu, verilen değeri ondalıklı bir sayıya ('double') çevirir.

'double' tipi, yüksek hassasiyetli hesaplamalar için tercih edilir.



Convert Kullanım Örneği (Kod)

Örnek: `int sayi = Convert.ToInt32(txtSayi1.Text);`

Bu komut, 'txtSayi1' adlı metin kutusundaki string içeriği alır ve onu 'sayi' isimli tamsayı değişkenine atar.



Alternatif Dönüşüm Yöntemi

Sayısal veri tiplerinin kendilerine ait .Parse() metotları da bulunmaktadır. Bu metotlar, Convert sınıfı ile işlevsel olarak benzerdir ve tipi doğrudan hedefleyerek dönüşümü gerçekleştirir.



Parse Kullanım Örneği (1/2)

Örnek: `int sayi = int.Parse(txtSayi.Text);`

Bu kullanımda, 'int' tipi doğrudan çağrılır ve string ifadeyi kendi formatına dönüştürmeye çalışır.

Parse Kullanım Örneği (2/2)

Örnek: `double sayi = double.Parse(txtSayi2.Text);`

Bu, ondalıklı sayı beklenen durumlarda 'double' veri tipi üzerinden dönüşümü sağlar.

Programın Kırılğanlığı (1/2)

Programlar, beklenmeyen veya hatalı kullanıcı girdilerine karşı 'kırılğan'dır. Bu durumlar, hatalara (Exceptions) yol açarak programın normal akışını bozar ve çökmesine neden olabilir.



Programın Kırılğanlığı (2/2)

Kullanıcı, sayı beklenen bir TextBox'a sayı yerine 'abc' gibi metin girerse ne olur?

Convert veya Parse metotları bu metni sayıya çeviremeyeceği için bir 'FormatException' hatası oluşur.

Program Çökmesi

FormatException gibi işlenmemiş hatalar, programın aniden durmasına ve çökmesine neden olur.
Bu, kullanıcı deneyimini olumsuz etkiler ve profesyonel yazılım standartlarına aykırıdır.



Savunmacı Programlama Felsefesi

Geliştiricilere, programlarının her zaman beklenen girdiyi almayabileceği felsefesi aşılmalıdır.
Uygulamalar, kullanıcıların 'beklenmedik şeyler' yapabileceği varsayımıyla 'savunmacı' bir yaklaşımla geliştirilmelidir.



Hata Yönetimine Giriş

try-catch yapısı, programın çökmesini engelleyen temel bir hata yönetim mekanizmasıdır.

Bu yapı, riskli kodların güvenli bir şekilde yürütülmesini sağlar.

Sigorta Poliçesi Analojisi

try-catch yapısı, bir sigorta poliçesine benzetilebilir.
'try' riskin alındığı, 'catch' ise risk gerçekleştiğinde devreye giren telafi mekanizmasıdır.



try Bloğu (1/2)

try {... } bloğu, hata verme potansiyeli yüksek olan 'riskli' kodları barındırır. Veri tipi dönüşümü gibi girdi bağımlı işlemler bu bloğa yazılmalıdır.

try Bloğu (2/2)

Program, try bloğu içindeki kodları çalıştırmayı 'dener'.
Eğer bu deneme sırasında başarılı olursa, catch bloğu atlanır ve program normal akışına devam eder.

catch Bloğu (1/2)

Eğer try bloğu içinde herhangi bir hata (exception) oluşursa, program çökmez. Bunun yerine, kontrol mekanizması otomatik olarak catch {... } bloğuna atlar.

catch Bloğu (2/2)

catch bloğunda, hata oluştuğunda çalışacak kodlar bulunur. Bu kodlar genellikle kullanıcıya durumu açıklayan bilgilendirici bir hata mesajı göstermek için kullanılır.



try-catch Genel Yapısı

C# dilinde try-catch yapısı aşağıdaki temel formatı izler:

```
try
{
    // Hata verebilecek kodlar
}
catch
{
    // Hata durumunda çalışacak kodlar
}
```

try Bloğu Detaylı Kod Örneği

```
try
{
double not1 = Convert.ToDouble(txtVize.Text);
// Diğer riskli dönüşümler veya işlemler...
}
```

Bu alanda vize notunun string'den double'a çevrilmesi hedeflenir.

catch Bloğu Detaylı Kod Örneği

```
catch  
{  
    MessageBox.Show(Lütfen geçerli bir sayı giriniz!);  
}
```

Hata oluştuğunda, kullanıcıya neyi yanlış yaptığına dair açıklayıcı bir mesaj gösterilir.

Uygulama Örneđi: Proje Fikri

Haftalık proje fikri olarak try-catch yapısını kullanarak bir Not Ortalama Hesaplayıcı geliştirilecektir.
Bu uygulama, programın hatalara karşı direncini test etmek için idealdir.



Form Tasarımı Bileşenleri (1/2)

Uygulama için 'Vize Notu:', 'Final Notu:' ve 'Ortalama:' başlıkları Label kontrolleri ile sağlanır.

Not girişleri için iki adet TextBox kontrolü (txtVize, txtFinal) eklenir.

Form Tasarımı Bileşenleri (2/2)

Sonucun gösterilmesi için bir Label kontrolü (lblOrtalama) kullanılır. Hesaplama işlemini başlatmak için 'Hesapla' isminde bir Button (btnHesapla) forma eklenir.



btnHesapla_Click Olayı

btnHesapla butonunun Click olayına tüm hesaplama mantığı yazılacaktır. Kullanıcı hatası riski nedeniyle tüm bu mantık, bir try-catch yapısı içine alınmalıdır.



Uygulama Kod Yapısı: try Bloğu (1/4)

```
try  
{  
double vizeNotu = Convert.ToDouble(txtVize.Text);  
Bu satır, kullanıcının girdiği metin veriyi ondalıklı sayıya dönüştürmeye çalışır.
```

Uygulama Kod Yapısı: try Bloğu (2/4)

```
double finalNotu = Convert.ToDouble(txtFinal.Text);
```

Benzer şekilde, final notu metin kutusundaki girdi de sayısal tipe dönüştürülür.

Uygulama Kod Yapısı: try Bloğu (3/4)

Ortalama hesaplaması için standart bir ağırlıklandırma kullanılır: Vizenin %40'ı, Finalin %60'ı.

```
double ortalama = (vizeNotu * 0.4) + (finalNotu * 0.6);
```

Uygulama Kod Yapısı: try Bloğu (4/4)

Hesaplanan sayısal ortalama değeri, tekrar string tipine dönüştürülerek kullanıcı arayüzüne (Label'a) yansıtılır.

```
lblOrtalama.Text = ortalama.ToString();
```



Uygulama Kod Yapısı: catch Bloğu (1/2)

```
catch  
{  
    MessageBox.Show(Hatalı giriş! Lütfen sadece sayısal değerler giriniz.);  
    Bu mesaj, bir FormatException veya benzeri bir dönüşüm hatası oluştuğunda  
    devreye girer.
```

Uygulama Kod Yapısı: catch Bloğu (2/2)

Hata oluştuğunda, kullanıcıya hesaplama yapılamadığını bildirmek önemlidir.
lblOrtalama.Text = Hesaplanamadı;
Bu, uygulamanın durumunu netleştirir.

Hata Yönetimi Felsefesi

try-catch yapısı, kullanıcı dostu ve stabil (sağlam) uygulamalar geliştirmenin temelini oluşturur.

Bu, programın hata durumlarında bile işlevselliğini sürdürmesini sağlar.



Kullanıcı Davranışı ve Önlemler

Öğrencilere, kullanıcıların her zaman beklenmedik değerler girebileceği felsefesi aşılmalıdır.
Programlama süreçlerinde bu riskleri önceden tahmin etmek temel bir adımdır.



Test Senaryosu 1

Program, geçerli sayısal girdilerle test edilmelidir (Örn: Vize: 50, Final: 80).
Bu senaryoda try bloğu başarılı olur ve ortalama doğru şekilde hesaplanır.

Test Senaryosu 2

Program, geçersiz girdilerle test edilmelidir (Örn: Vize: abc, Final: 80).
Bu senaryoda Convert metodu FormatException üretir ve kontrol catch bloğuna geçer.



Test Senaryosu 3

Girdi alanlarının boş bırakılması durumunda da programın davranışı gözlemlenmelidir.
Boş stringin sayıya dönüşümü de FormatException'a yol açacaktır.

try-catch'in Faydaları

Yapılan testler sonucunda try-catch bloğunun, hatalı girdilere rağmen programın tamamen çökmesini nasıl engellediği gözlemlenir. Bu, kesintisiz bir kullanıcı deneyimi sağlar.



String Veri Tiplerinin Özellikleri

String veri tipi, alfanümerik karakter dizilerini tutar. Programlamada, kullanıcı arayüzü girdileri standart olarak bu tipte kabul edilir. Bu tipler, matematiksel işlem operatörlerini doğrudan uygulayamaz.



Sayısal Tipler: Int vs. Double

Int (ToInt32), sadece tam sayıları tutar. Double ise ondalıklı değerleri yüksek hassasiyetle saklar.

Ortalama hesaplama gibi ondalık sonuç beklentisi olan işlemlerde double kullanılması akademik olarak daha doğrudur.



Convert ve Parse: Derinlemesine Farklar

Convert sınıfı, genel bir sınıftır ve birçok tip arasında dönüşüm yapabilir. Null değerleri de yönetebilir.

Parse metotları ise, hedef veri tipinin kendi static metotlarıdır ve genellikle string'den o tipe dönüşüm için kullanılır.

32-bit Tamsayı Formatı

`Convert.ToInt32()` metodu, 32 bitlik ikili sayı sistemini kullanan bir tamsayı formatına dönüştürme gerçekleştirir.
Bu, yaklaşık -2.1 milyardan +2.1 milyara kadar değer aralığını temsil eder.

Hata Yönetiminde Mesaj Kutusu

catch bloğu içinde kullanılan `MessageBox.Show()`, program çökmek yerine kullanıcıya anlaşılır bir pop-up mesajı sunar. Bu, hatanın nedenini belirtmek ve kullanıcıyı doğru girişe yönlendirmek için kritiktir.



Kod Parçalama Tekniđi

Sadece hata verme potansiyeli olan kod satırlarının try blođu içine alınması önerilir.
Risk içermeyen kodlar, try-catch yapısının dışında tutularak performans artışı sağlanabilir.



İstisna Mekanizmasının Akademik Önemi

İstisna yönetimi (Exception Handling), modern yazılım mühendisliğinde kodun dayanıklılığını ve sürdürülebilirliğini artıran temel bir disiplindir.



Veri Giriş Kontrolü

try-catch, bir son savunma hattıdır. Mümkünse, TextBox'lara sadece sayı girilmesini sağlayan önleyici kontroller (maskeleye veya klavye olayları) de eklenmelidir.



Final Notunun Ağırlığı

Uygulama örneğinde final notuna %60 ağırlık verilmiştir. Bu, final sınavının öğrenci başarısı üzerindeki etkisini modelleyen tipik bir akademik değerlendirme yöntemidir.



Vize Notunun Ağırlığı

Vize notunun %40 ağırlıkta olması, öğrencinin dönem içindeki başarısının belirli bir oranda değerlendirmeye katıldığını gösterir.



Dönüşüm Hatalarının Sınıflandırılması

FormatException, bir metodun beklediği formatta olmayan bir girdiyi işlemesi sonucu oluşur.
Örneğin, sayısal dönüşüm metodunun metin ile karşılaşması bu hatayı tetikler.



Veri Geri Alma Stratejisi

catch bloğu, yalnızca mesaj göstermekle kalmaz. Programın güvenli bir duruma dönmesini, örneğin hatalı giriş alanını sıfırlamasını sağlayacak kodları da içerebilir.



Görsel Programlama ve Arayüz Etkileşimi

Görsel programlamada, kullanıcı etkileşimleri genellikle buton tıklama (Click) olayları ile yönetilir.

Bu olaylar içinde veri dönüşümü, arayüzden alınan verinin arka planda işlenmesi için elzemdir.



Girdi Veri Bütünlüğü

Güvenilir bir uygulama için girdi verilerinin bütünlüğü kritiktir. try-catch mekanizması, bu bütünlüğün sağlanamadığı durumlarda sistemin stabilitesini korur.



Double.ToString() Metodu

Hesaplanan double tipindeki 'ortalama' değerinin kullanıcı arayüzü Label'ına atanabilmesi için tekrar string'e çevrilmesi gerekir. ToString() metodu bu dönüşümü standart formatta gerçekleştirir.



MessageBox Kullanımında Kullanıcı Dili

catch bloğunda verilen hata mesajları teknik olmamalıdır. Kullanıcının anlayacağı dilde, sorunun ne olduğunu ve nasıl çözüleceğini belirten ifadeler kullanılmalıdır.



Program Akışı Kontrolü

Normal bir program akışında, try bloğu bittikten sonra catch bloğu atlanır. Hata oluştuğunda ise try bloğu kesilir ve akış catch bloğuna yönlendirilir. Bu yönlendirme, programın çökme noktasını es geçer.



try-catch Uygulama Kapsamı

try-catch sadece veri dönüşüm hataları için değil, aynı zamanda dosya okuma/yazma, ağ bağlantısı veya veritabanı erişimi gibi riskli tüm I/O işlemlerinde kullanılır.



Convert Sınıfı ve Type Safety

Convert sınıfı, dönüşümü yaparken kaynağın tipini kontrol eder ve uygun olmayan durumlarda istisna fırlatır. Bu, dönüşümün kontrollü bir şekilde yapılmasını sağlar.



Parse Metodunun Sınırlılıkları

Parse metotları, genellikle null (boş) bir string girişini doğrudan işleyemez ve bu durumda `ArgumentNullException` veya `FormatException` fırlatabilir. `Convert` bu konuda daha esnek davranır.



Hata Mesajında Detay Verme

Akademik uygulamalarda, genel 'catch' yerine spesifik hata tiplerinin (örneğin `catch (FormatException e)`) yakalanması ve hatanın detaylarının loglanması daha ileri düzey bir yaklaşımdır.



Defansif Programlamanın Kritikliđi

Programlamada defansif stratejilerin benimsenmesi, yazılımın uzun ömürlü, bakımı kolay ve son kullanıcının hatalarına karşı dayanıklı olmasını sağlar.



Veri Dönüşümünde Kültür Etkisi

Ondalık sayıların (double) dönüşümünde, farklı kültürlerde kullanılan ondalık ayırıcıların (virgül veya nokta) dikkate alınması gerekmektedir. Convert ve Parse metotları bu lokalizasyon ayarlarından etkilenebilir.



Görsel Programlamada Kod Yerleşimi

Veri işleme ve hata yönetimi mantığının, kullanıcı arayüzü kodundan ayrı bir yerde tutulması (buton olayları içinde değil de ayrı metotlarda) daha temiz bir kod yapısı oluşturur.



Veri Akış Şeması

TextBox (string) -> Convert/Parse (riskli) -> try -> double (sayısal) -> Hesaplama -> ToString() (string) -> Label (arayüz).

Önemli Not: Güvenlik ve try-catch

Güvenlik açısından, catch bloklarında programın iç işleyişine dair detaylı teknik hataların (stack trace) son kullanıcıya gösterilmemesi esastır. Yalnızca kullanıcı dostu mesajlar sunulmalıdır.



Özet: Veri Dönüşümü Neden Önemli?

Veri dönüşümü, metinsel girdileri matematiksel veya mantıksal olarak işlenebilir formata getiren zorunlu bir adımdır. Olmadan programın temel fonksiyonları yerine getirilemez.



Özet: Hata Yönetimi Neden Önemli?

Hata yönetimi, kullanıcıların yanlışlıkla veya kasten hatalı girdi vermesi durumunda uygulamanın kararlılığını korur. Bu, profesyonel bir yazılımın en temel gereksinimidir.



Görsel Programlamada, Convert veya Parse gibi riskli dönüşüm metotlarını try bloğu içinde konumlandırmak, haftanın temel öğretisidir.



Sonuç ve Değerlendirme

Bu hafta öğrenilen try-catch yapısı, üniversite öğrencilerinin daha güvenilir, kullanıcı odaklı ve endüstri standartlarına uygun yazılımlar geliştirmesi için kritik bir araçtır.

Başarılı bir programlama, sadece çalışan değil, aynı zamanda hatalara dayanıklı olan koddur.



Veri Dönüşümü

Süleyman EZDEMİR

suleyman.ezdemir@giresun.edu.tr

